

LA(1)LR(0)

임기정

Abstract

This note fixes the LR(0) characteristic automaton, defines the one-symbol lookahead set attached to each completed LR(0) item, and derives the usual Read/Follow digraph computation. The proofs are written so that the essential claims appear as inclusions, fixed-point equations, and derivation chains.

The LR(0) Automaton

Definition 1 ($\text{LRA}(G)$). Let $G \triangleq (N, T, P, S)$ be a context-free grammar. Define:

(i)

$$G' = (N', T', P', S') \triangleq (N \uplus \{S'\}, T \uplus \{\$, \}, P \cup \{[S' \rightarrow S\$]\}, S').$$

(ii)

$$V \triangleq N \uplus T, \quad V' \triangleq N' \uplus T'.$$

(iii)

$$\beta \xRightarrow{\text{rm}}_{P'} \gamma \quad \xLeftrightarrow{\text{def}} \quad (\beta, \gamma) \in \{(\alpha Az, \alpha \omega z) \mid [A \rightarrow \omega] \in P', \alpha \in V'^*, z \in T'^*\}.$$

(iv)

$$I_{\text{LR}(0)} \triangleq \{[A \rightarrow \alpha \cdot \beta] \mid [A \rightarrow \alpha\beta] \in P'\}.$$

(v) For $q \subseteq I_{\text{LR}(0)}$,

$$\text{Cl}(q) =_{\mu} q \cup \{[A \rightarrow \varepsilon \cdot \omega] \mid [A \rightarrow \omega] \in P', [B \rightarrow \beta \cdot A\gamma] \in \text{Cl}(q)\}.$$

(vi)

$$q_0 \triangleq \text{Cl}(\{[S' \rightarrow \varepsilon \cdot S\$]\}).$$

(vii) For $q \subseteq I_{\text{LR}(0)}$ and $X \in V'$,

$$\text{GOTO}(q, X) \triangleq \text{Cl}(\{[A \rightarrow \alpha X \cdot \beta] \mid [A \rightarrow \alpha \cdot X\beta] \in q\}).$$

(viii)

$$Q \triangleq \text{PT} \setminus \{\emptyset\}, \quad \text{PT} =_{\mu} \{q_0\} \cup \{\text{GOTO}(q, X) \mid q \in \text{PT}, X \in V'\}.$$

(ix) The path relation is the least relation satisfying

$$\varepsilon : p \rightarrow q \iff p \in Q \wedge p = q, \quad X\alpha : p \rightarrow q \iff p \in Q \wedge \alpha : \text{GOTO}(p, X) \rightarrow q.$$

(x)

$$\mathbf{Config} \triangleq \{(\alpha : p \rightarrow q, z) \mid \alpha \in V'^*, p, q \in Q, z \in T'^*, \alpha : p \rightarrow q\}.$$

(xi)

$$\delta(q, X) \triangleq \text{GOTO}(q, X) \quad (q \in Q, X \in V').$$

(xii)

$$\mathbf{reduce}(q, t) \triangleq \{[A \rightarrow \omega] \mid [A \rightarrow \omega \cdot \varepsilon] \in q\} \quad (q \in Q, t \in T').$$

(xiii) Let $\vdash \subseteq \mathbf{Config} \times \mathbf{Config}$ be generated by:

$$\frac{q'' = \delta(q', t)}{(\alpha : q \rightarrow q', tz) \vdash (\alpha t : q \rightarrow q'', z)} \text{ Shift}$$

$$\frac{\alpha : q \rightarrow p \quad \omega : p \rightarrow q' \quad [A \rightarrow \omega] \in \mathbf{reduce}(q', t) \quad q'' = \delta(p, A)}{(\alpha\omega : q \rightarrow q', tz) \vdash (\alpha A : q \rightarrow q'', tz)} \text{ Reduce}(A \rightarrow \omega)$$

(xiv)

$$q_f \triangleq \delta(\delta(q_0, S), \$).$$

(xv) The accepted language is

$$L(\text{LRA}(G)) \triangleq \{z \in T^* \mid (\varepsilon : q_0 \rightarrow q_0, z\$) \vdash^* (S\$: q_0 \rightarrow q_f, \varepsilon)\}.$$

Fact 2 (Correctness of $\text{LRA}(G)$).

$$L(G) = L(\text{LRA}(G)), \quad L(G) \triangleq \{z \in T^* \mid S \Rightarrow_P^* z\}.$$

Proof. For $c = (\alpha : p \rightarrow q, u) \in \mathbf{Config}$, put

$$Y(c) \triangleq \alpha u \in V'^*.$$

$\text{LR}(0)$ PATH FACTS. For $\eta \in V'^*$, the characteristic automaton satisfies

$$(\exists q \in Q) \eta : q_0 \rightarrow q \iff \eta \text{ is a viable prefix of } G', \quad (1)$$

$$\left. \begin{array}{l} \eta = \eta' \omega, \quad \eta' : q_0 \rightarrow p, \quad \omega : p \rightarrow q, \\ S' \xRightarrow[\mathbf{rm}_{P'}]{*} \eta' A y \xRightarrow[\mathbf{rm}_{P'}]{*} \eta' \omega y, \quad y = tz \in T'^* \end{array} \right\} \implies [A \rightarrow \omega \cdot \varepsilon] \in q \wedge [A \rightarrow \omega] \in \mathbf{reduce}(q, t). \quad (2)$$

These are the standard inductive invariants of Cl, GOTO, and the path relation in Definition 1.

STEP INVARIANT. Each parser step reverses zero or one rightmost grammar step:

$$c \vdash c' \implies \begin{cases} Y(c') = Y(c), & \text{if the step is Shift,} \\ Y(c') \xRightarrow[\mathbf{rm}_{P'}]{*} Y(c), & \text{if the step is Reduce.} \end{cases} \quad (3)$$

Indeed,

$$\begin{aligned} \text{Shift : } Y(\alpha : q \rightarrow q', tz) &= \alpha tz = Y(\alpha t : q \rightarrow q'', z), \\ \text{Reduce : } Y(\alpha A : q \rightarrow q'', tz) &= \alpha A tz \xRightarrow[\mathbf{rm}_{P'}]{*} \alpha \omega tz = Y(\alpha \omega : q \rightarrow q', tz). \end{aligned}$$

Thus

$$c \vdash^* c' \implies Y(c') \xRightarrow[\mathbf{rm}_{P'}]{*} Y(c). \quad (4)$$

SOUNDNESS.

$$\begin{aligned} z \in L(\text{LRA}(G)) &\implies (\varepsilon : q_0 \rightarrow q_0, z\$) \vdash^* (S\$: q_0 \rightarrow q_f, \varepsilon) \\ &\xRightarrow{(4)} S\$ \xRightarrow[\mathbf{rm}_{P'}]{*} z\$ \\ &\implies S \Rightarrow_P^* z \\ &\implies z \in L(G). \end{aligned}$$

The third implication deletes the endmarker: S' never occurs on a right-hand side, so $[S' \rightarrow S\$]$ is unused in a derivation from $S\$$, and $\$$ is never rewritten.

COMPLETENESS. Assume $S \Rightarrow_P^* z$. Choose a rightmost derivation in G' :

$$S\$ = \gamma_0 \xrightarrow{\text{rm}}_{P'} \gamma_1 \xrightarrow{\text{rm}}_{P'} \cdots \xrightarrow{\text{rm}}_{P'} \gamma_N = z\$.$$
 (5)

Write its k -th step as

$$\gamma_k = \alpha_k A_k y_k \xrightarrow{\text{rm}}_{P'} \alpha_k \omega_k y_k = \gamma_{k+1}, \quad [A_k \rightarrow \omega_k] \in P', \quad y_k = t_k z_k \in T'^*.$$
 (6)

By (2), for each k there exist $p_k, q_k \in Q$ such that

$$\alpha_k : q_0 \rightarrow p_k, \quad \omega_k : p_k \rightarrow q_k, \quad [A_k \rightarrow \omega_k \cdot \varepsilon] \in q_k, \quad [A_k \rightarrow \omega_k] \in \mathbf{reduce}(q_k, t_k). \quad (7)$$

Reading (5) from right to left gives, for $k = N - 1, \dots, 0$,

$$(\alpha_k \omega_k : q_0 \rightarrow q_k, y_k) \vdash (\alpha_k A_k : q_0 \rightarrow \delta(p_k, A_k), y_k), \quad (8)$$

with any intervening terminal symbols consumed by Shift steps along the viable prefix path. Iterating (8) and then shifting the endmarker yields

$$(\varepsilon : q_0 \rightarrow q_0, z\$) \vdash^* (S\$: q_0 \rightarrow q_f, \varepsilon), \quad q_f = \delta(\delta(q_0, S), \$).$$

Hence $z \in L(\text{LRA}(G))$. Together with soundness, this proves the claim. $\square$

Lookahead Restriction

Theorem 3 (Lookahead-guarded reductions). *Define*

$$\mathbf{LA} : \{(q, [A \rightarrow \omega]) \mid q \in Q, [A \rightarrow \omega \cdot \varepsilon] \in q\} \longrightarrow \mathcal{P}(T')$$

by

$$\mathbf{LA}(q, [A \rightarrow \omega]) \triangleq \left\{ t \in T' \mid S' \xrightarrow{\text{rm}}_{P'}^* \alpha A t z, \alpha \omega : q_0 \rightarrow q, \alpha \in V'^*, z \in T'^* \right\}. \quad (9)$$

Replace the $\text{LR}(0)$ reduction function by

$$\mathbf{reduce}_{\mathbf{LA}}(q, t) \triangleq \{[A \rightarrow \omega] \mid [A \rightarrow \omega \cdot \varepsilon] \in q, t \in \mathbf{LA}(q, [A \rightarrow \omega])\}. \quad (10)$$

If the resulting table has no shift/reduce or reduce/reduce conflict, it is an $\text{LALR}(1)$ parser and accepts $L(G)$.

Proof. Let $\vdash_{\mathbf{LA}}$ be the transition relation obtained from $\vdash$ by replacing $\mathbf{reduce}$ with $\mathbf{reduce}_{\mathbf{LA}}$. From (10),

$$\mathbf{reduce}_{\mathbf{LA}}(q, t) \subseteq \mathbf{reduce}(q, t) \implies \vdash_{\mathbf{LA}} \subseteq \vdash \implies \vdash_{\mathbf{LA}}^* \subseteq \vdash^*.$$

Therefore

$$L(\vdash_{\mathbf{LA}}) \subseteq L(\text{LRA}(G)) = L(G) \quad \text{by Fact 2.} \quad (11)$$

Conversely, let $z \in L(G)$. In the completeness construction of Fact 2, every Reduce step has the form

$$(\alpha \omega : q_0 \rightarrow q, t z') \vdash (\alpha A : q_0 \rightarrow \delta(p, A), t z'),$$

where

$$S' \xrightarrow{\text{rm}}_{P'} S\$ \xrightarrow{\text{rm}}_{P'}^* \alpha A t z', \quad \alpha \omega : q_0 \rightarrow q, \quad [A \rightarrow \omega \cdot \varepsilon] \in q.$$

By (9),

$$t \in \mathbf{LA}(q, [A \rightarrow \omega]) \iff [A \rightarrow \omega] \in \mathbf{reduce}_{\mathbf{LA}}(q, t).$$

Thus every Reduce step used in that accepting $\text{LR}(0)$ computation is also legal for $\vdash_{\mathbf{LA}}$; Shift steps are unchanged. Hence

$$L(G) \subseteq L(\vdash_{\mathbf{LA}}).$$

Combining this with (11) gives $L(\vdash_{\mathbf{LA}}) = L(G)$.

Finally, the action at (q, t) consists of the shift $\delta(q, t)$ when it is nonempty and the reductions in $\mathbf{reduce}_{\mathbf{LA}}(q, t)$. Absence of shift/reduce and reduce/reduce conflicts means that this action is single-valued under one-symbol lookahead, which is precisely the $\text{LALR}(1)$ table condition. $\square$

Read and Follow Sets

Theorem 4 (Digraph computation of lookahead). *Let*

$$D \triangleq \{(p, A) \mid p \in Q, A \in N', \delta(p, A) \neq \emptyset\}.$$

For a relation R , write

$$R!x \triangleq \{y \mid (x, y) \in R\}.$$

Define **Read**, **Follow** $\subseteq D \times T'$ as the least relations generated by the rules

$$\begin{array}{c} \frac{\delta(\delta(p, A), t) \neq \emptyset}{t \in \mathbf{Read}!(p, A)} \text{DR} \qquad \frac{\delta(p, A) = r \quad C \Rightarrow_{P'}^* \varepsilon \quad (r, C) \in D}{\mathbf{Read}!(r, C) \subseteq \mathbf{Read}!(p, A)} \text{reads} \\[10pt] \frac{t \in \mathbf{Read}!(p, A)}{t \in \mathbf{Follow}!(p, A)} \text{Read} \quad \frac{[B \rightarrow \beta \cdot A\gamma] \in p \quad \beta : p' \rightarrow p \quad \gamma \Rightarrow_{P'}^* \varepsilon \quad (p', B) \in D}{\mathbf{Follow}!(p', B) \subseteq \mathbf{Follow}!(p, A)} \text{includes} \end{array}$$

Then, whenever $q \in Q$ and $[A \rightarrow \omega \cdot \varepsilon] \in q$,

$$\mathbf{LA}(q, [A \rightarrow \omega]) = \{t \in T' \mid \exists p \in Q. \omega : p \rightarrow q \wedge \delta(p, A) \neq \emptyset \wedge t \in \mathbf{Follow}!(p, A)\}. \quad (12)$$

Proof. Define the semantic read and follow sets on D :

$$\text{Read}(p, A) \triangleq \{t \in T' \mid \exists r, s, \gamma. r = \delta(p, A) \wedge \gamma \Rightarrow_{P'}^* \varepsilon \wedge \gamma t : r \rightarrow s\}, \quad (13)$$

$$\text{Follow}(p, A) \triangleq \left\{ t \in T' \mid \exists \alpha, z. S' \xrightarrow[\text{rm}]{*}_{P'} \alpha A t z \wedge \alpha : q_0 \rightarrow p \wedge z \in T'^* \right\}. \quad (14)$$

Read = Read. For $x = (p, A) \in D$, set

$$\text{DR}(p, A) \triangleq \{t \in T' \mid \delta(\delta(p, A), t) \neq \emptyset\}.$$

The two **Read**-rules define the least fixed point of the monotone operator

$$\Phi_R(F)(p, A) = \text{DR}(p, A) \cup \bigcup_{\substack{\delta(p, A) = r \\ C \Rightarrow_{P'}^* \varepsilon \\ (r, C) \in D}} F(r, C).$$

By (13), $t \in \text{Read}(p, A)$ iff for some nullable $\gamma = C_1 \cdots C_n \in N'^*$,

$$\delta(p, A) = r_0, \quad \delta(r_{i-1}, C_i) = r_i \quad (1 \leq i \leq n), \quad \delta(r_n, t) \neq \emptyset.$$

Equivalently,

$$t \in \underbrace{\Phi_R^n(\emptyset)(p, A)}_{n \text{ reads edges, then DR}},$$

and hence

$$\mathbf{Read}!(p, A) = \mu \Phi_R(p, A) = \text{Read}(p, A) \quad ((p, A) \in D). \quad (15)$$

Follow = Follow. The two **Follow**-rules define the least fixed point of

$$\Phi_F(F)(p, A) = \text{Read}(p, A) \cup \bigcup_{\substack{[B \rightarrow \beta \cdot A\gamma] \in p \\ \beta : p' \rightarrow p \\ \gamma \Rightarrow_{P'}^* \varepsilon \\ (p', B) \in D}} F(p', B).$$

First Follow is closed under Φ_F . For the Read-seed:

$$\begin{aligned} t \in \text{Read}(p, A) &\implies \exists \alpha, \gamma, z. \alpha : q_0 \rightarrow p \wedge \gamma \Rightarrow_{P'}^* \varepsilon \wedge S' \xRightarrow[\text{rm}_{P'}]^* \alpha A \gamma t z \\ &\implies S' \xRightarrow[\text{rm}_{P'}]^* \alpha A \gamma t z \xRightarrow[\text{rm}_{P'}]^* \alpha A t z \\ &\implies t \in \text{Follow}(p, A). \end{aligned}$$

For an includes edge,

$$\begin{aligned} t \in \text{Follow}(p', B), \quad [B \rightarrow \beta \cdot A \gamma] \in p, \quad \beta : p' \rightarrow p, \quad \gamma \Rightarrow_{P'}^* \varepsilon \Big\} &\implies S' \xRightarrow[\text{rm}_{P'}]^* \alpha' B t z \\ &\xRightarrow[\text{rm}_{P'}]^* \alpha' \beta A \gamma t z \\ &\xRightarrow[\text{rm}_{P'}]^* \alpha' \beta A t z, \end{aligned}$$

with $\alpha' \beta : q_0 \rightarrow p$; hence

$$\text{Follow}(p', B) \subseteq \text{Follow}(p, A).$$

Conversely, let $t \in \text{Follow}(p, A)$, witnessed by

$$S' \xRightarrow[\text{rm}_{P'}]^* \alpha A t z, \quad \alpha : q_0 \rightarrow p.$$

Looking at the production that created this occurrence of A gives a factor

$$S' \xRightarrow[\text{rm}_{P'}]^* \alpha' B t' z' \xRightarrow[\text{rm}_{P'}]^* \alpha' \beta A \gamma t' z' \xRightarrow[\text{rm}_{P'}]^* \alpha A t z, \quad \alpha = \alpha' \beta.$$

Thus exactly one of the following alternatives holds:

$$\begin{cases} \gamma = \gamma_0 t \gamma_1, & \gamma_0 \Rightarrow_{P'}^* \varepsilon, & t \in \text{Read}(p, A), \\ \gamma \Rightarrow_{P'}^* \varepsilon, & & t \in \text{Follow}(p', B) \text{ and } (p, A) \rightsquigarrow_{\text{includes}} (p', B). \end{cases}$$

In the second case the witnessing rightmost derivation is strictly shorter. Induction on that length gives

$$\text{Follow}(p, A) \subseteq \mu \Phi_F(p, A).$$

Together with closure,

$$\mathbf{Follow}!(p, A) = \mu \Phi_F(p, A) = \text{Follow}(p, A) \quad ((p, A) \in D). \quad (16)$$

LOOKAHEAD. For $q \in Q$ and $[A \rightarrow \omega \cdot \varepsilon] \in q$,

$$\begin{aligned} t \in \mathbf{LA}(q, [A \rightarrow \omega]) &\iff \exists \alpha, z. S' \xRightarrow[\text{rm}_{P'}]^* \alpha A t z \wedge \alpha \omega : q_0 \rightarrow q \\ &\iff \exists p, \alpha, z. S' \xRightarrow[\text{rm}_{P'}]^* \alpha A t z \wedge \alpha : q_0 \rightarrow p \wedge \omega : p \rightarrow q \\ &\iff \exists p. \omega : p \rightarrow q \wedge \delta(p, A) \neq \emptyset \wedge t \in \text{Follow}(p, A) \\ &\stackrel{(16)}{\iff} \exists p. \omega : p \rightarrow q \wedge \delta(p, A) \neq \emptyset \wedge t \in \mathbf{Follow}!(p, A). \end{aligned}$$

The side condition $\delta(p, A) \neq \emptyset$ is automatic for $A \neq S'$: tracing $[A \rightarrow \omega \cdot \varepsilon] \in q$ backward along $\omega : p \rightarrow q$ gives $[A \rightarrow \varepsilon \cdot \omega] \in p$, which is introduced by an item with the dot before A . For $A = S'$, both sides are empty because S' occurs on no right-hand side and the parser does not reduce $[S' \rightarrow S\$]$. This is (12). $\square$

A Digraph Algorithm and Implementation Specification

This appendix gives an executable specification for computing the sets used in Theorem 4. The construction is the usual digraph view of LALR(1) lookahead computation: direct seed sets are placed on nodes, dependency edges describe set inclusions, and the least fixed point is obtained by collapsing strongly connected components and propagating set unions along the resulting DAG [1, 2, 3].

A.1 General Digraph Fixed-Point Problem

Let X be a finite set of nodes, let $E \subseteq X \times X$ be a directed edge relation, and let $\text{Seed} : X \rightarrow \mathcal{P}(T')$. We use the edge orientation

$$x \rightsquigarrow y \quad \text{to mean} \quad F(y) \subseteq F(x).$$

Thus the desired solution is the least function $F : X \rightarrow \mathcal{P}(T')$ satisfying

$$F(x) = \text{Seed}(x) \cup \bigcup_{x \rightsquigarrow y} F(y). \quad (17)$$

Algorithm DIGRAPH(X, E, Seed).

1. Compute the strongly connected components of (X, E) , for example by Tarjan's depth-first-search algorithm.
2. Build the condensation DAG $\mathcal{C}$. Its nodes are SCCs, and $C \rightsquigarrow D$ is an edge when some $x \in C$ and $y \in D$ satisfy $x \rightsquigarrow y$, with $C \neq D$.
3. Initialize, for every SCC C ,

$$\text{Val}(C) \leftarrow \bigcup_{x \in C} \text{Seed}(x).$$

4. Process the SCCs in reverse topological order with respect to $\rightsquigarrow$, so every successor D of C has already been processed, and put

$$\text{Val}(C) \leftarrow \text{Val}(C) \cup \bigcup_{C \rightsquigarrow D} \text{Val}(D).$$

5. Return $F(x) \triangleq \text{Val}(C_x)$, where C_x is the SCC containing x .

Correctness condition. For every SCC C , after step 4,

$$\text{Val}(C) = \bigcup \{ \text{Seed}(y) \mid \text{some } x \in C \text{ reaches } y \text{ in } (X, E) \}.$$

Therefore the returned F is exactly the least solution of (17). This is the only property of the graph algorithm needed by Theorem 4.

Complexity. Let $b = \lceil |T'|/w \rceil$, where w is the machine word size used for bitsets. SCC construction is $O(|X| + |E|)$, and set propagation is

$$O((|X| + |E|)b).$$

A simple work-list implementation that repeatedly applies

$$F(x) := F(x) \cup F(y) \quad (x \rightsquigarrow y)$$

is also correct, but the SCC version avoids repeated propagation inside cycles.

A.2 Instance for Read

First compute nullable nonterminals by the least fixed point

$$\text{Null}(A) \iff \exists [A \rightarrow X_1 \cdots X_k] \in P' \text{ such that every } X_i \text{ is nullable,}$$

where terminals are never nullable and $k = 0$ is allowed. Extend this to strings by

$$\text{Null}(X_1 \cdots X_k) \iff \bigwedge_{i=1}^k \text{Null}(X_i).$$

For $x = (p, A) \in D$, define the direct-read seed

$$\text{DR}(p, A) \triangleq \{t \in T' \mid \delta(\delta(p, A), t) \neq \emptyset\}.$$

Define $E_R \subseteq D \times D$ by

$$(p, A) \rightsquigarrow_R (r, C) \iff \delta(p, A) = r \wedge C \in N' \wedge \text{Null}(C) \wedge (r, C) \in D.$$

If $(r, C) \notin D$, then $\mathbf{Read}!(r, C) = \emptyset$, so no edge is needed. Compute

$$\mathbf{Read}!(p, A) \triangleq \text{DIGRAPH}(D, E_R, \text{DR})(p, A).$$

A.3 Instance for Follow

The seed for follow propagation is the already computed read set:

$$\text{Seed}_F(p, A) \triangleq \mathbf{Read}!(p, A).$$

Define $E_I \subseteq D \times D$ by

$$(p, A) \rightsquigarrow_I (p', B) \iff [B \rightarrow \beta \cdot A\gamma] \in p \wedge \beta : p' \rightarrow p \wedge \text{Null}(\gamma) \wedge (p', B) \in D.$$

The orientation means

$$\mathbf{Follow}!(p', B) \subseteq \mathbf{Follow}!(p, A).$$

Compute

$$\mathbf{Follow}!(p, A) \triangleq \text{DIGRAPH}(D, E_I, \text{Seed}_F)(p, A).$$

A.4 Lookback Relation and Lookahead Construction

For every completed item $[A \rightarrow \omega \cdot \varepsilon] \in q$, define

$$\text{LB}(q, A \rightarrow \omega) \triangleq \{(p, A) \in D \mid \omega : p \rightarrow q\}.$$

Then

$$\mathbf{LA}(q, [A \rightarrow \omega]) \triangleq \bigcup_{(p, A) \in \text{LB}(q, A \rightarrow \omega)} \mathbf{Follow}!(p, A),$$

which is (12) written as an implementation step.

Implementation note. The query $\omega : p \rightarrow q$ can be implemented by following δ -edges from p along ω . The query $\beta : p' \rightarrow p$ in E_I can be implemented by caching reverse-path queries keyed by (β, p) , or by storing predecessor edges during construction of the LR(0) automaton.

A.5 Parser-Table and Conflict Specification

After $\mathbf{LA}$ has been computed, construct actions as follows.

1. If $\delta(q, t) \neq \emptyset$ for $t \in T'$, add the shift action

$$q \xrightarrow{t} \delta(q, t).$$

2. For each $[A \rightarrow \omega \cdot \varepsilon] \in q$ and each $t \in \mathbf{LA}(q, [A \rightarrow \omega])$, add the reduce action $A \rightarrow \omega$ under lookahead t .
3. Report a shift/reduce conflict when both a shift action and a reduce action are present for the same pair (q, t) .

4. Report a reduce/reduce conflict when two distinct reduce productions are present for the same pair (q, t) .

The accepting condition remains the one in Definition 1: the parser accepts when it reaches

$$(S\$: q_0 \rightarrow q_f, \varepsilon).$$

Equivalently, in a conventional ACTION/GOTO table, q_f may be treated as the accepting state after the input marker has been consumed.

A.6 Summary Specification

The complete computation required by Theorem 4 is:

1. Build G' , Q , and δ as in Definition 1.
2. Compute nullable nonterminals and nullable strings.
3. Build D , DR , and E_R ; compute **Read** by $\text{DIGRAPH}(D, E_R, DR)$.
4. Build E_I ; compute **Follow** by $\text{DIGRAPH}(D, E_I, \mathbf{Read})$.
5. Build LB and compute each $\mathbf{LA}(q, [A \rightarrow \omega])$ by unioning the appropriate **Follow**-sets.
6. Construct the guarded reduce actions and check conflicts.

The postcondition is that the computed **Read**, **Follow**, and **LA** are the least sets satisfying Theorem 4, and hence the lookahead-restricted parser of Theorem 3 uses exactly the LALR(1) lookahead sets determined by the LR(0) automaton.

References

- [1] F. DeRemer and T. J. Pennello. Efficient computation of LALR(1) look-ahead sets. *ACM Transactions on Programming Languages and Systems*, 4(4):615–649, 1982.
- [2] R. E. Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972.
- [3] J. H. Gallier. A survey of LR-parsing methods: The graph method for computing fixed points and LALR(1) lookahead sets. CIS 511 lecture notes, University of Pennsylvania, 2008.